1. Project Title

Self-Healing and Scalable Web Infrastructure on AWS using Application Load Balancer and Auto Scaling

2. Abstract

Modern web applications must stay available even when servers fail or traffic changes suddenly. Traditional systems depend on manual monitoring and intervention, which leads to downtime, slow recovery and wasted resources. This project builds a self-healing, auto-scaling web infrastructure on Amazon Web Services (AWS). Ubuntu-based Amazon EC2 instances run an Nginx web server and are launched from a Launch Template. An Application Load Balancer distributes incoming traffic across the instances through a Target Group and continuously performs health checks. An Auto Scaling Group maintains the desired number of instances, replaces failed ones, and adds or removes instances according to CPU utilization. Amazon CloudWatch monitors metrics and triggers scaling alarms, Amazon S3 stores static assets and backups, and an IAM Role gives EC2 secure access to S3 without hard-coded credentials. The purpose of the project is to demonstrate high availability, fault tolerance and cost-efficient scaling with minimal human effort. The expected outcome is a web application that remains online during instance failures, scales out under heavy load and scales in when demand falls, reducing cost.

3. Introduction

Cloud computing lets organizations rent computing resources on demand instead of maintaining physical servers. A common challenge is keeping applications available while traffic varies and failures occur. A single server is a single point of failure, and fixed capacity is either too small at peak times or wasteful at quiet times. AWS addresses this with Elastic Load Balancing, Auto Scaling and CloudWatch, which together let infrastructure monitor its own health, recover automatically (self-healing) and adjust capacity to demand (elasticity). This project applies these concepts in a simple, low-cost, student-friendly architecture. It matters because high availability, resilience and cost optimization are core requirements of real-world cloud systems.

4. Key Points of the Project

- Web application served by **Nginx** on **Ubuntu** EC2 instances, each page showing its hostname/private IP
- **Application Load Balancer** as the single public entry point, spreading traffic across Availability Zones
- **Health checks** that detect failed instances automatically
- **Self-healing:** the Auto Scaling Group replaces terminated or unhealthy instances
- **CPU-based scaling policy** for automatic scale-out and scale-in (min / desired / max capacity)
- **Launch Template + User Data** for fully automated, repeatable instance setup
- **Security:** EC2 accepts HTTP only from the ALB Security Group; IAM Role used instead of access keys
- **CloudWatch** dashboards and alarms; **S3** (private) for static assets and backups

5. Advantages & Limitations

Advantages	Limitations
• High availability: no single point of failure	• Scaling is not instant (boot time and cooldown)
• Self-healing with zero manual intervention	• ALB and extra instances add cost if left running
• Elastic: scales out for load, scales in to save cost	• Instances must be stateless; data needs S3/RDS/EFS
• Better performance through load distribution	• Simple CPU-only policy may not fit all workloads
• Consistent, automated, repeatable deployments	• HTTP only in the demo (no HTTPS/custom domain)
• Secure: Security Group chaining and IAM Roles	• Needs careful threshold tuning and AWS knowledge

6. Components in the Diagram

The flowchart below shows how a request travels through the system (top row) and how monitoring, self-healing and Auto Scaling react to load and failures (lower rows).

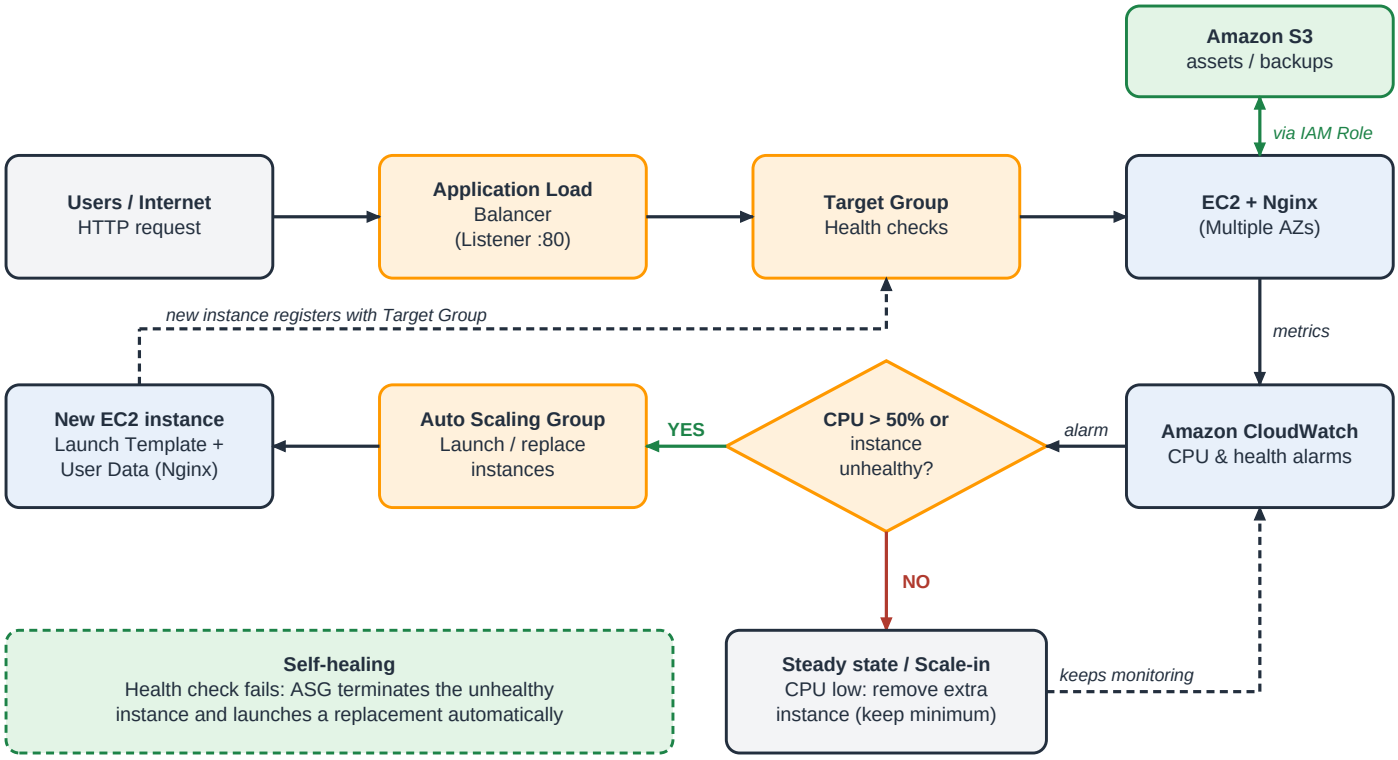

Figure 1: Workflow of the self-healing and auto-scaling infrastructure

Component	Role in the system
Users / Internet	Clients that send HTTP requests to the application through the ALB DNS name.
Application Load Balancer	Public entry point (Layer 7). Its listener on port 80 forwards requests to healthy targets.
Target Group	Registered pool of EC2 instances; runs health checks and only routes to healthy ones.
EC2 + Nginx (Ubuntu)	Application servers in different Availability Zones serving the website and showing instance ID.
Launch Template	Blueprint (AMI, instance type, IAM Role, Security Group, User Data) used to create instances.
Auto Scaling Group	Maintains min/desired/max capacity, replaces unhealthy instances and applies scaling policy.
Amazon CloudWatch	Collects CPU and health metrics, raises alarms that trigger scale-out and scale-in.
Amazon S3	Private bucket for static assets and backups, accessed by EC2 through its IAM Role.
IAM Role	Gives EC2 temporary permissions to S3, so no access keys are stored on servers.
VPC, Subnets, Security Groups	Network layer: public subnets in 2+ AZs; EC2 allows HTTP only from the ALB Security Group.

7. Future Scope

- Enable **HTTPS** using AWS Certificate Manager and a custom domain with Route 53
- Add a **database** (Amazon RDS / Aurora Multi-AZ) for dynamic, stateful applications
- Use **private subnets with NAT**, plus CloudFront and AWS WAF for stronger security
- Adopt **target-tracking policies** on request count and set up SNS email alerts
- Automate deployment with **Terraform / CloudFormation** and a CI/CD pipeline

8. Conclusion

The project demonstrates that combining EC2, an Application Load Balancer, Auto Scaling and CloudWatch produces an infrastructure that heals itself and adapts to demand without manual effort. Failed instances are replaced automatically, traffic is shared across healthy servers, and capacity grows or shrinks with CPU load, giving high availability, good performance and lower cost. It also gives a practical, beginner-friendly foundation in core cloud concepts that can be extended to production systems.